

BITP 3113: OBJECT ORIENTED PROGRAMMING

Mini Project Deliverable

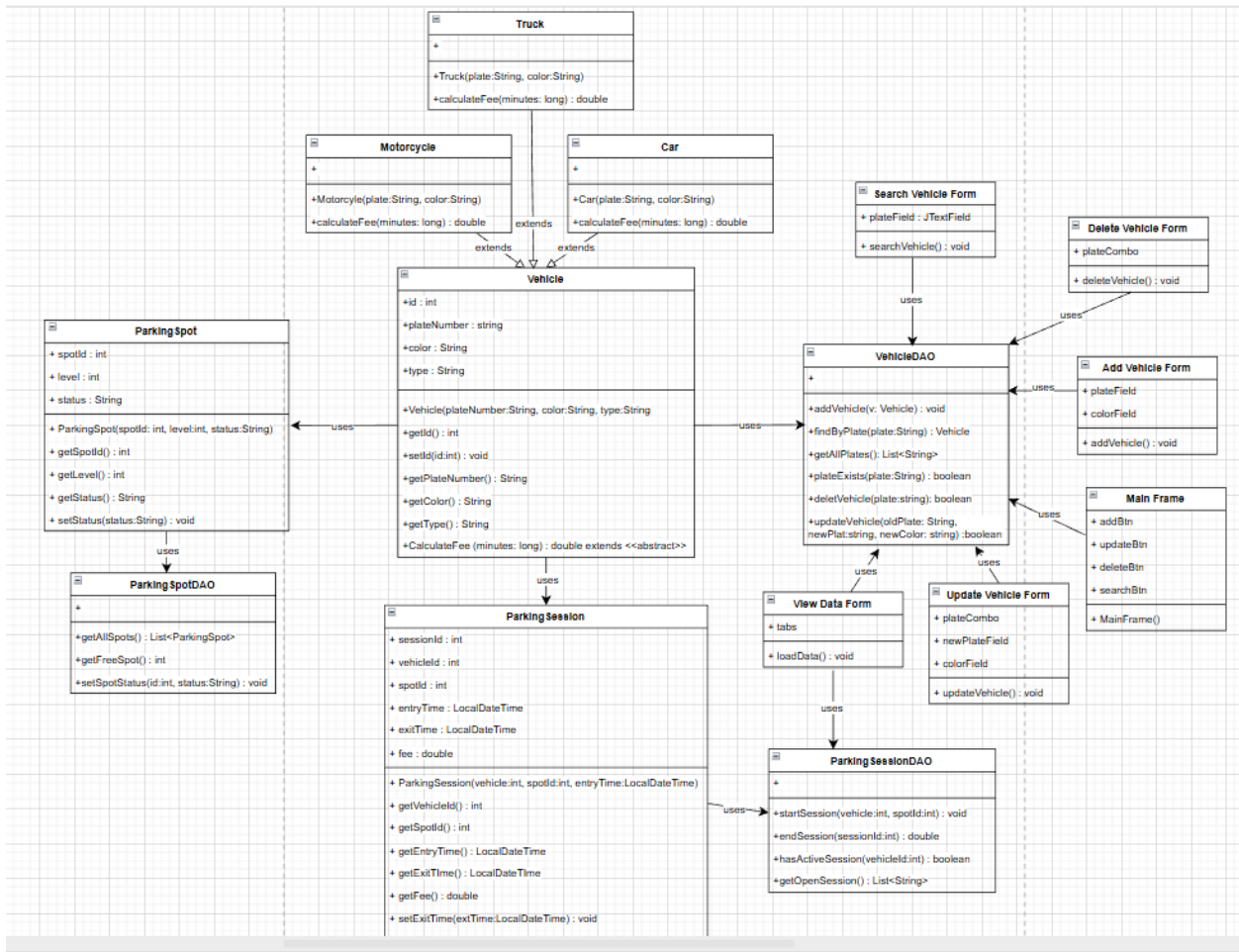
Project Title: Car Parking Management System

Team members: 4

Matric	Name	Section/ Group
B032410993	HUSSAM ALDIN MAHMOUD	S2G2
B032410994	KHALED ANANARA	S2G2
B032410852	NIK I'ZZ ISKANDAR ZULQARNAIN BIN JUSLAN	S2G2
B032410395	MUHAMMAD FAUZAN FAIZ BIN RAMLEE	S2G2

Diagram:

1) UML diagrams

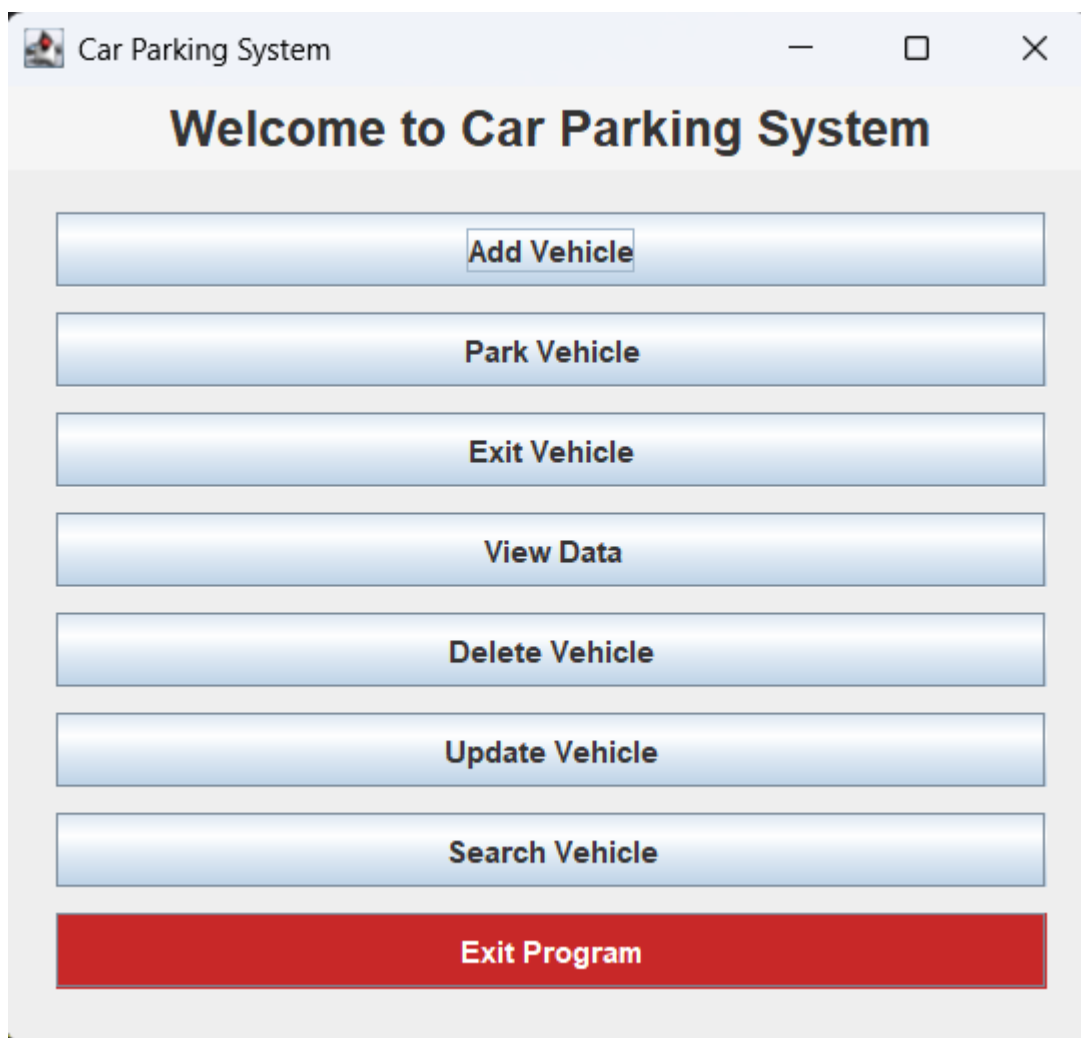


This UML diagram shows the structure of the Car Parking System including inheritance, DAO classes, and relationships between classes

2) GUI

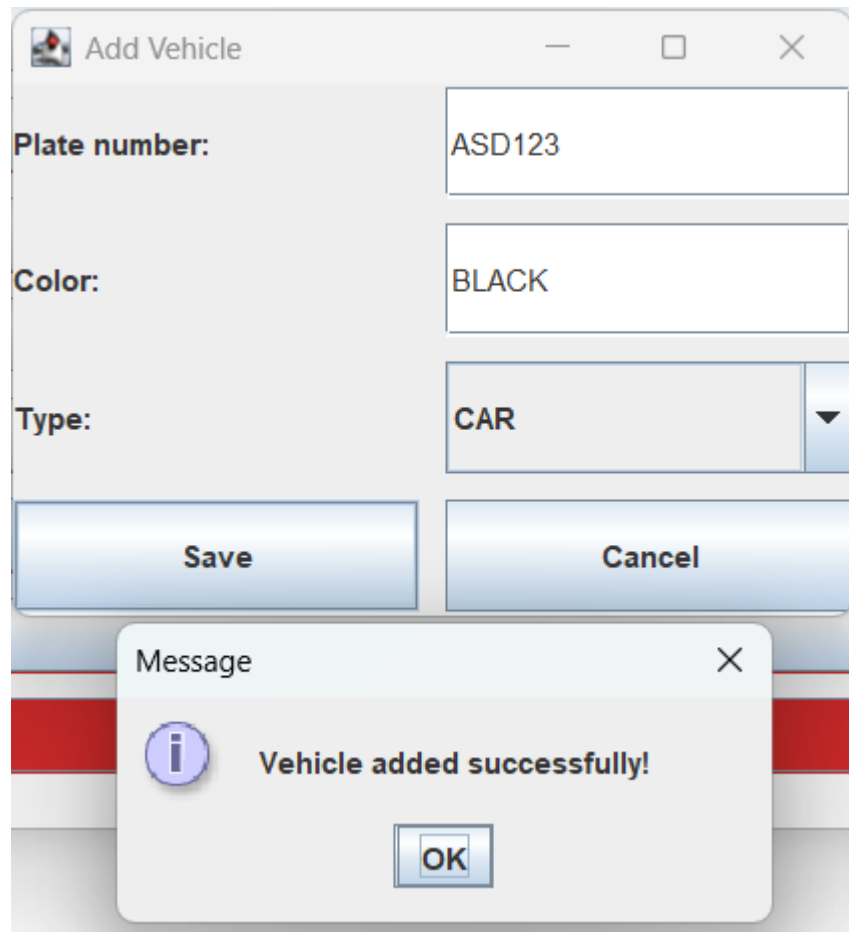
Main Menu

This is the main menu of the Car Parking System. It allows the user to navigate to different system features



Add Vehicle Screen

This GUI allows the user to add a new vehicle by entering the plate number, color, and selecting the vehicle type. The data will be stored in the system



The image shows a Windows-style application window titled "Add Vehicle". It contains three input fields: "Plate number:" with the value "ASD123", "Color:" with the value "BLACK", and "Type:" with a dropdown menu showing "CAR". Below these fields are two buttons: "Save" and "Cancel". In the foreground, a "Message" dialog box is displayed, featuring an information icon and the text "Vehicle added successfully!". An "OK" button is at the bottom of the message box.

Plate number:	ASD123
Color:	BLACK
Type:	CAR

Save Cancel

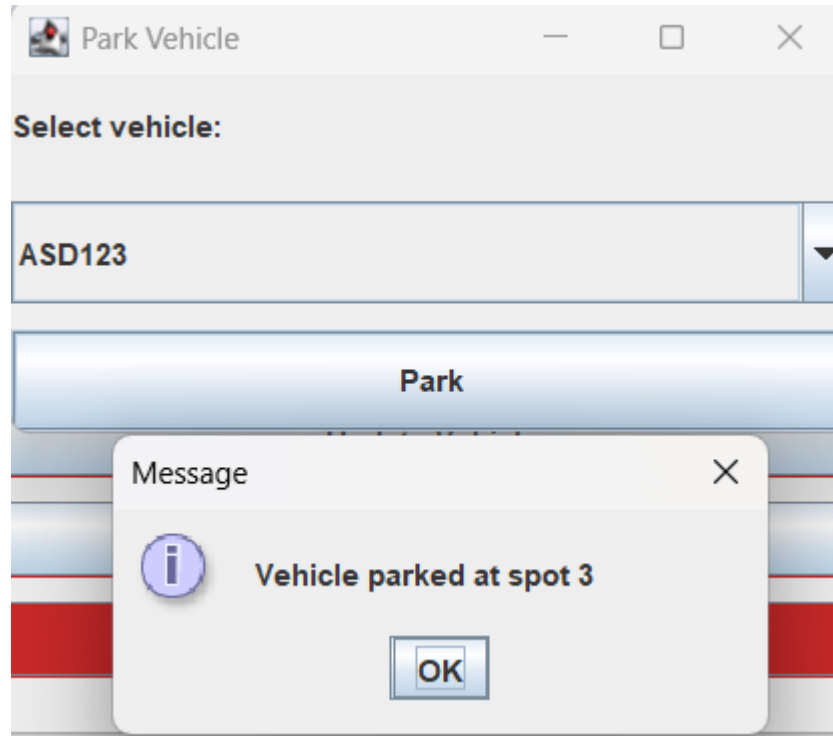
Message

i Vehicle added successfully!

OK

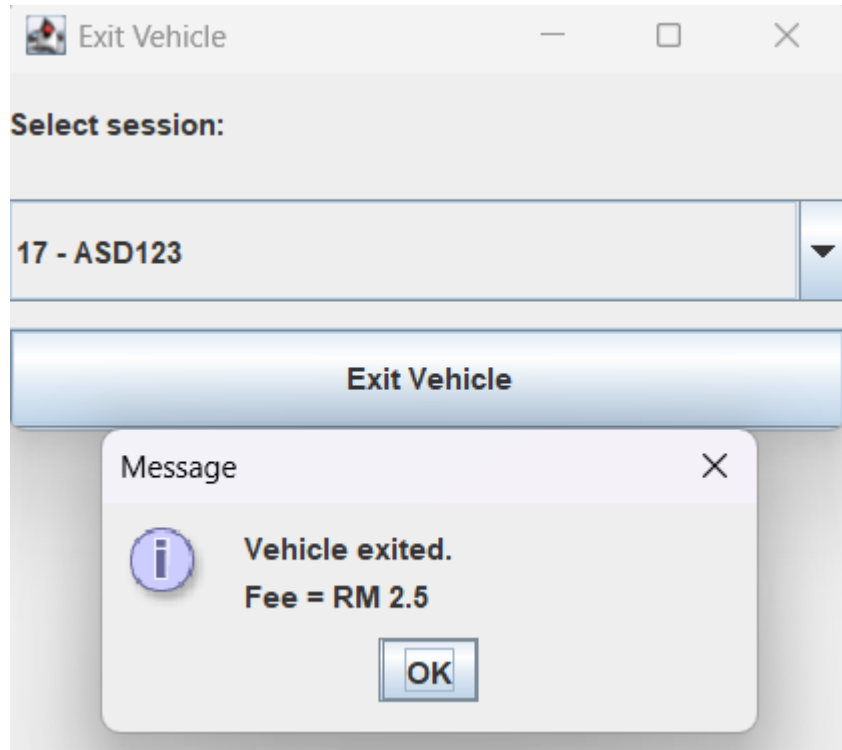
Park Vehicle Screen

This interface allows the user to select a registered vehicle and start a parking session. The selected vehicle will be recorded together with the entry time



Exit Vehicle Screen

This screen allows the user to select an active parking session and end it. When the session is closed, the system calculates the parking fee based on the parking duration



View Data Screen

This interface displays the stored data in the system. The user can view vehicles, parking spots, and parking sessions using different tabs

View Data			
Vehicles Parking Spots Sessions			
vehicle_id	plate_number	type	color
1	ABC321	CAR	Red
2	MOTO55	MOTORCYCLE	Black
3	TRK300	TRUCK	Blue
4	KLM221	CAR	White
5	CAR777	CAR	yellow
7	TRUCK8	TRUCK	Grey
9	ZONE44	CAR	Blue
10	RIDE12	MOTORCYCLE	Orange
11	zxc123	CAR	read
12	ASD123	CAR	BLACK

View Data		
Vehicles Parking Spots Sessions		
spot_id	level	status
1	1	OCCUPIED
2	1	OCCUPIED
3	1	FREE
4	1	FREE
5	1	FREE
6	2	FREE
7	2	FREE
8	2	FREE
9	2	FREE
10	2	FREE
11	3	FREE
12	3	FREE
13	3	FREE
14	3	FREE
15	3	FREE

View Data					
Vehicles		Parking Spots		Sessions	
session_id	vehicle_id	spot_id	entry_time	exit_time	fee
12	1	1	2026-01-02T18:...	2026-01-02T18:4...	0.55
13	1	1	2026-01-06T13:...	2026-01-06T13:5...	0.5
14	11	1	2026-01-06T14:...	2026-01-06T14:3...	1.95
15	1	2	2026-01-06T14:...		
16	5	1	2026-01-06T14:...		
17	12	3	2026-01-11T18:...	2026-01-11T18:2...	2.5

Delete Vehicle

This screen allows the user to delete an existing vehicle from the system by selecting its plate number from the list.

Delete Vehicle

Select vehicle plate:

Delete Cancel

Confirm Delete

Are you sure you want to delete KLM221?

Yes No

Message

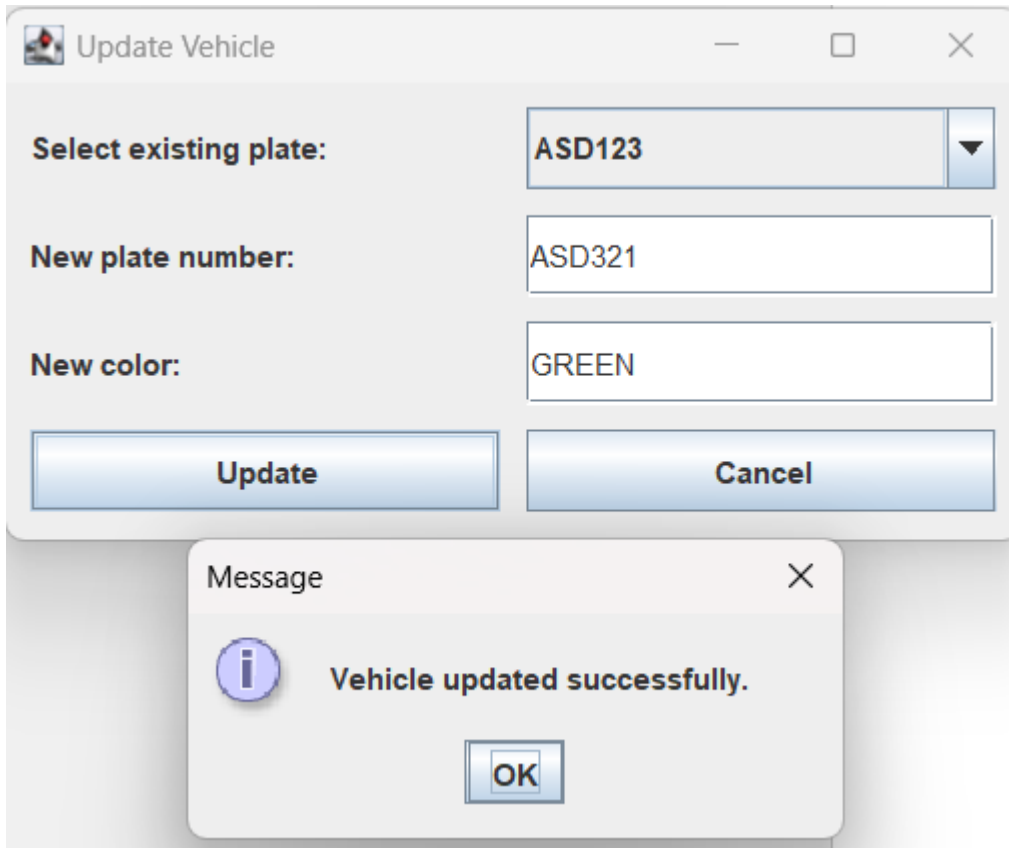
Vehicle deleted successfully.

OK

Update:

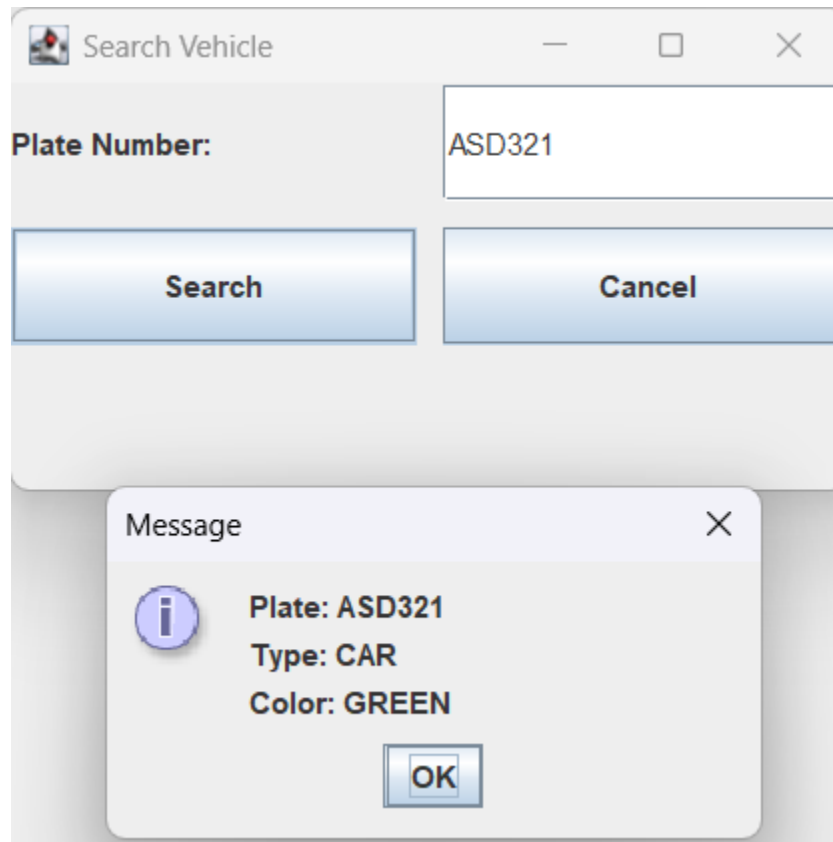
This screen allows the user to update existing vehicle information.

The user selects a vehicle by its plate number and updates the plate or color, and the system confirms the update successfully.



The image displays a software interface with two windows. The top window, titled 'Update Vehicle', contains three input fields: 'Select existing plate:' with a dropdown menu showing 'ASD123', 'New plate number:' with a text box containing 'ASD321', and 'New color:' with a text box containing 'GREEN'. Below these fields are two buttons: 'Update' and 'Cancel'. The bottom window, titled 'Message', features an information icon (a lowercase 'i' in a circle) and the text 'Vehicle updated successfully.' Below this text is an 'OK' button.

Search



The image shows a software interface with two windows. The top window, titled 'Search Vehicle', has a text input field containing 'ASD321' and two buttons labeled 'Search' and 'Cancel'. The bottom window, titled 'Message', displays search results: 'Plate: ASD321', 'Type: CAR', and 'Color: GREEN', with an 'OK' button at the bottom.

Search Vehicle

Plate Number: ASD321

Search **Cancel**

Message

i **Plate: ASD321**
Type: CAR
Color: GREEN

OK

Code snapshot:

1) Inheritance / Abstract

In this project, the class `Vehicle` is declared as an abstract class. It contains common attributes and an abstract method that must be implemented by all subclasses

```
1 package model;
2
3 public abstract class Vehicle {
4     protected int id;
5     protected String plateNumber;
6     protected String color;
7     protected String type;
8
9     public int getId() { return id; }
10    public void setId(int id) { this.id = id; }
11
12
13    public Vehicle(String plateNumber, String color, String type) {
14        this.plateNumber = plateNumber;
15        this.color = color;
16        this.type = type;
17    }
18
19    public abstract double calculateFee(long minutes);
20
21    public String getPlateNumber() { return plateNumber; }
22    public String getColor() { return color; }
23    public String getType() { return type; }
24 }
25
26
```

```

1 package model;
2
3 public class Car extends Vehicle {
4
5     public Car(String plate, String color) {
6         super(plate, color, "CAR");
7     }
8
9
10    public double calculateFee(long minutes) {
11        return (minutes / 60.0) * 2.0;
12    }
13 }
14
15

```

2) Polymorphism

Polymorphism is implemented by using a Vehicle reference to represent different objects such as Car, Motorcycle, and Truck.

The actual object type is determined at runtime based on the vehicle type retrieved from the database.

```

Vehicle v = null;

switch (type) {
    case "CAR": v = new Car(plate, color); break;
    case "MOTORCYCLE": v = new Motorcycle(plate, color); break;
    case "TRUCK": v = new Truck(plate, color); break;
}

v.setId(id);
return v;
}

```

3) Interface

The interface is designed to define a common set of operations for vehicle management. Although it is not directly referenced in the current implementation, it is used to demonstrate the concept of abstraction and to support future scalability and flexibility of the system.

4) Java Collection: List / Set / Map

```
public List<String> getAllPlates() throws Exception {  
  
    String sql = "SELECT plate_number FROM vehicle";  
  
    try (Connection con = DatabaseConnection.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql);  
        ResultSet rs = ps.executeQuery()) {  
  
        List<String> list = new java.util.ArrayList<>();  
  
        while (rs.next())  
            list.add(rs.getString(1));  
  
        return list;  
    }  
}
```

The system uses a List collection to store and return multiple vehicle plate numbers retrieved from the database.

A Map collection can be used to store data in key-value pairs, such as mapping vehicle plate numbers to their corresponding vehicle objects. This approach can improve data retrieval efficiency in future system enhancements.

5) Exception Handling (Create your own)

```
1 package dao;
2
3 public class DuplicatePlateException extends Exception {
4
5     public DuplicatePlateException(String message) {
6         super(message);
7     }
8 }
9
10
```

```
// Custom Exception Handling
if (plateExists(v.getPlateNumber())) {
    throw new DuplicatePlateException("Plate number already exists.");
}
```

A custom exception is created to handle application-specific errors.

(DuplicatePlateException) is thrown when attempting to add a vehicle with an existing plate number, preventing duplicate data and improving system reliability.

6) Connecting to database(Insert, view, search, update, delete)

Insert:

```
public void addVehicle(Vehicle v) throws Exception {  
    // Custom Exception Handling  
    if (plateExists(v.getPlateNumber())) {  
        throw new DuplicatePlateException("Plate number already exists.");  
    }  
  
    String sql = "INSERT INTO vehicle (plate_number, type, color) VALUES (?, ?, ?)";  
  
    try (Connection con = DatabaseConnection.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql)) {  
  
        ps.setString(1, v.getPlateNumber());  
        ps.setString(2, v.getType());  
        ps.setString(3, v.getColor());  
  
        ps.executeUpdate();  
    }  
}
```

This code inserts a new vehicle record into the database using SQL INSERT and **PreparedStatement**.

The values are taken from the GUI and stored safely into the table

View Data

```
public class ViewDataForm extends JFrame {  
  
    JTabbedPane tabs;  
  
    public ViewDataForm() throws Exception {  
  
        setTitle("View Data");  
        setSize(600, 400);  
        setLocationRelativeTo(null);  
  
        tabs = new JTabbedPane();  
  
        tabs.addTab("Vehicles", createTable("SELECT * FROM vehicle"));  
        tabs.addTab("Parking Spots", createTable("SELECT * FROM parking_spot"));  
        tabs.addTab("Sessions", createTable("SELECT * FROM parking_session"));  
  
        add(tabs);  
    }  
  
    JPanel createTable(String sql) throws Exception {  
  
        JPanel panel = new JPanel(new BorderLayout());  
  
        Connection con = DatabaseConnection.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql);  
        ResultSet rs = ps.executeQuery();  
  
        JTable table = new JTable(buildTableModel(rs));  
  
        panel.add(new JScrollPane(table), BorderLayout.CENTER);  
  
        return panel;  
    }  
}
```

I used SELECT to read data from the database and display it in a table in the GUI.

Search

```
public Vehicle findByPlate(String plate) throws Exception {  
    String sql = "SELECT * FROM vehicle WHERE plate_number=?";  
    try (Connection con = DatabaseConnection.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql)) {  
        ps.setString(1, plate);  
        ResultSet rs = ps.executeQuery();  
        if (rs.next()) {  
            int id = rs.getInt("vehicle_id");  
            String type = rs.getString("type");  
            String color = rs.getString("color");  
            Vehicle v = null;  
            switch (type) {  
                case "CAR": v = new Car(plate, color); break;  
                case "MOTORCYCLE": v = new Motorcycle(plate, color); break;  
                case "TRUCK": v = new Truck(plate, color); break;  
            }  
            v.setId(id);  
            return v;  
        }  
        return null;  
    }  
}
```

The search function retrieves vehicle information from the database based on the plate number.

A parameterized SQL query is used to ensure safe and accurate searching.

Update

```
public boolean updateVehicle(String oldPlate, String newPlate, String newColor) throws Exception {  
    StringBuilder sql = new StringBuilder("UPDATE vehicle SET ");  
    boolean hasChange = false;  
  
    if (newPlate != null && !newPlate.isEmpty()) {  
        sql.append("plate_number = ?");  
        hasChange = true;  
    }  
  
    if (newColor != null && !newColor.isEmpty()) {  
        if (hasChange) sql.append(", ");  
        sql.append("color = ?");  
        hasChange = true;  
    }  
  
    if (!hasChange) return false;  
  
    sql.append(" WHERE plate_number = ?");  
  
    try (Connection con = DatabaseConnection.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql.toString())) {  
  
        int index = 1;  
  
        if (newPlate != null && !newPlate.isEmpty())  
            ps.setString(index++, newPlate);  
  
        if (newColor != null && !newColor.isEmpty())  
            ps.setString(index++, newColor);  
  
        ps.setString(index, oldPlate);  
  
        return ps.executeUpdate() > 0;  
    }  
}
```

The update functionality allows the system to modify existing vehicle information in the database. An SQL UPDATE statement is dynamically constructed and executed using **PreparedStatement** to safely update records.

Delete

```
public boolean deleteVehicle(String plate) throws Exception {  
  
    if (isVehicleParked(plate)) {  
        return false;  
    }  
  
    String sql = "DELETE FROM vehicle WHERE plate_number=?";  
  
    try (Connection con = DatabaseConnection.getConnection();  
        PreparedStatement ps = con.prepareStatement(sql)) {  
  
        ps.setString(1, plate);  
        int rows = ps.executeUpdate();  
  
        return rows > 0;  
    }  
}
```

The delete functionality removes a vehicle record from the database using an SQL DELETE statement.

The system checks whether the vehicle is currently parked before allowing deletion to maintain data consistency.